



Mobile Application System Design

(Chapter 4)

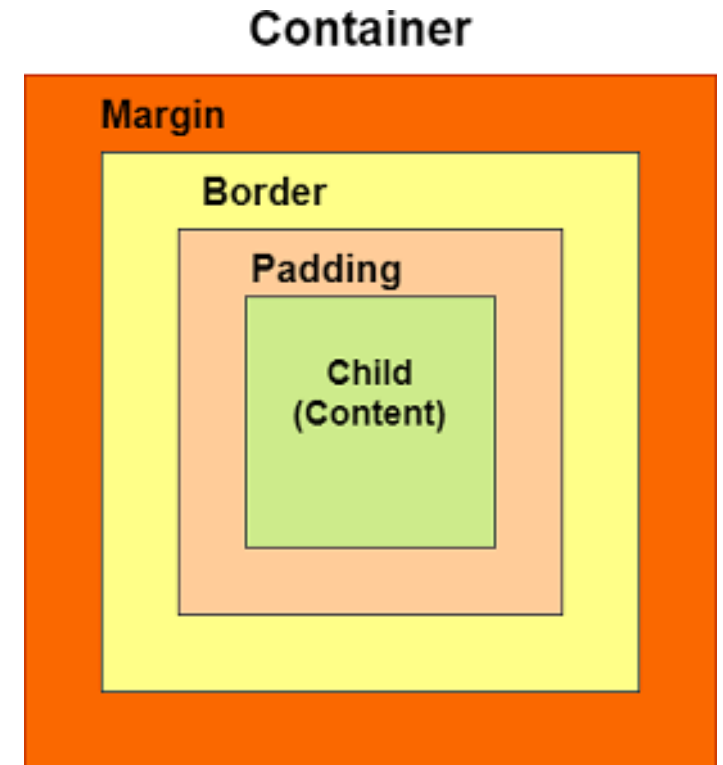
Course Lecturer: Nurul Amera Muhamad Nazmi

Key Concepts: Designing Custom Views

1. Nested Widgets in Container
2. ListView
3. GridView

Nested Widgets in Container

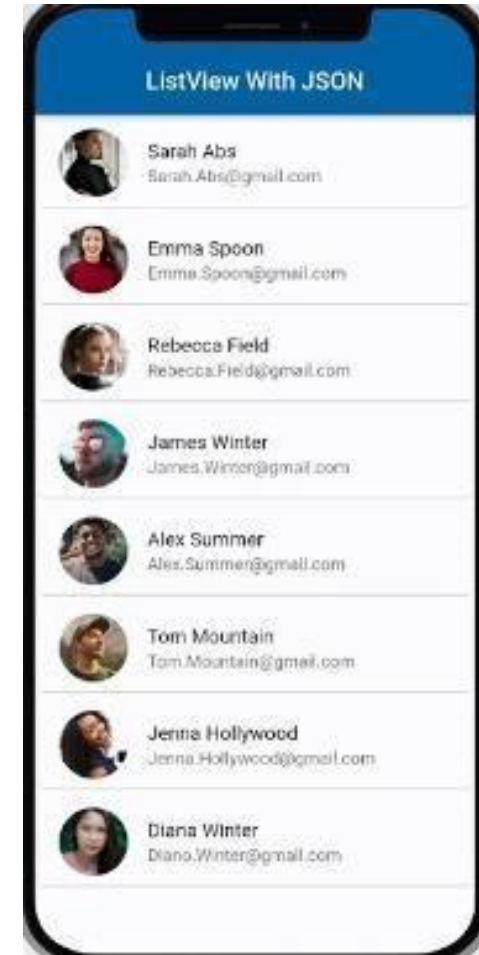
- Container in flutter is a convenience widget that combines common coloring, positioning, and sizing of widgets.
- A Container widget can be used to store one or more widgets and position them on the screen according to our convenience.
- Basically, a container is like a box to store contents.



CLASS ACTIVITY

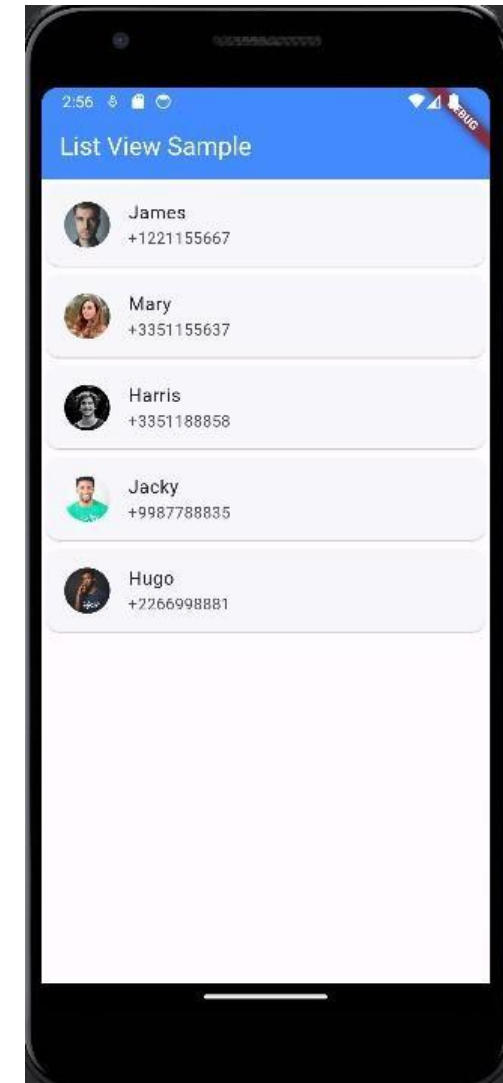
(1st Hands-On Practice)

- Flutter ListView is the most commonly used scrolling widget.
- It displays its children one after another in the scroll direction.



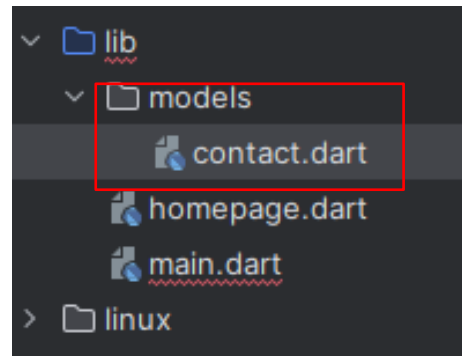
Example App (Contact List)

- In this Contact List app, we are going to build a simple contact list using ListView.
- Create a new Flutter project.
- Create a new folder and subfolder named “assets” and “images”
- Copy and paste the given people-1.jpg, people-2.jpg, ..., people-5.jpg to the images folder.
- Set the pubspec.yaml file for image path.



Example App (Contact List)

- Create a new subfolder named “models” inside the lib folder.
- Create a new dart file named “contact.dart” in the “models” subfolder.
- Create a Contact data class in the file.



```
main.dart  contact.dart x
1  class Contact{
2      String name;
3      String image;
4      String phoneNumber;
5
6      Contact(
7          {
8              required this.name,
9              required this.image,
10             required this.phoneNumber
11         }
12     );
13
14 }
```

Example App (Contact List)

- In Contact class, create a function named getAllContacts().
- This function should return a list of contacts.

```
Contact(  
    {  
        required this.name,  
        required this.image,  
        required this.phoneNumber  
    }  
);  
  
static List <Contact> getAllContacts(){  
    List <Contact> contacts = [];  
  
    return contacts;  
}  
}
```


Example App (Contact List)

- In the getAllContacts() function, add an Contact data object to the contacts list.

```
static List <Contact> getAllContacts(){  
    List <Contact> contacts = [];  
  
    contacts.add(  
        Contact(  
            name: "James",  
            image: "assets/images/people-1.jpg",  
            phoneNumber: "+1221155667"  
        )  
    );  
  
    return contacts;  
}
```

Example App (Contact List)

- Add the second data object to the contact list.

```
static List <Contact> getAllContacts(){  
    List <Contact> contacts = [];  
  
    contacts.add(  
        Contact(  
            name: "James",  
            image: "assets/images/people-1.jpg",  
            phoneNumber: "+1221155667"  
        )  
    );  
  
    contacts.add(  
        Contact(  
            name: "Mary",  
            image: "assets/images/people-2.jpg",  
            phoneNumber: "+3351155637"  
        )  
    );  
  
    return contacts;  
}
```

Example App (Contact List)

- Add the third, fourth and fifth data to the contact list by repeating the similar steps.

```
contacts.add(  
    Contact(  
        name: "Mary",  
        image: "assets/images/people-2.jpg",  
        phoneNumber: "+3351155637"  
    )  
);  
  
contacts.add(  
    Contact(  
        name: "Harris",  
        image: "assets/images/people-3.jpg",  
        phoneNumber: "+3351188858"  
    )  
);  
  
contacts.add(  
    Contact(  
        name: "Jacky",  
        image: "assets/images/people-4.jpg",  
        phoneNumber: "+9987788835"  
    )  
);  
  
contacts.add(  
    Contact(  
        name: "Hugo",  
        image: "assets/images/people-5.jpg",  
        phoneNumber: "+2266998881"  
    )  
);  
  
return contacts;  
}
```

Example App (Contact List)

- Create a new dart file named “homepage.dart”.
- In the new dart file, create a StatelessWidget
- Create a new variable named **contacts** to store a list of contact data.

```
main.dart  contact.dart  homepage.dart x
1  import 'package:flutter/material.dart';
2  import 'package:week_4_example_2/models/contact.dart';
3
4  class HomePage extends StatelessWidget {
5    HomePage({super.key});
6
7    List<Contact> contacts = [];
8
9    @override
10   Widget build(context){
11
12
13   }
14
15 }
```

Example App (Contact List)

- In the build function, invoke the getAllContacts() function to extract all the contact info and store them in the contacts variable.
- Set a Scaffold layout for the ListView.

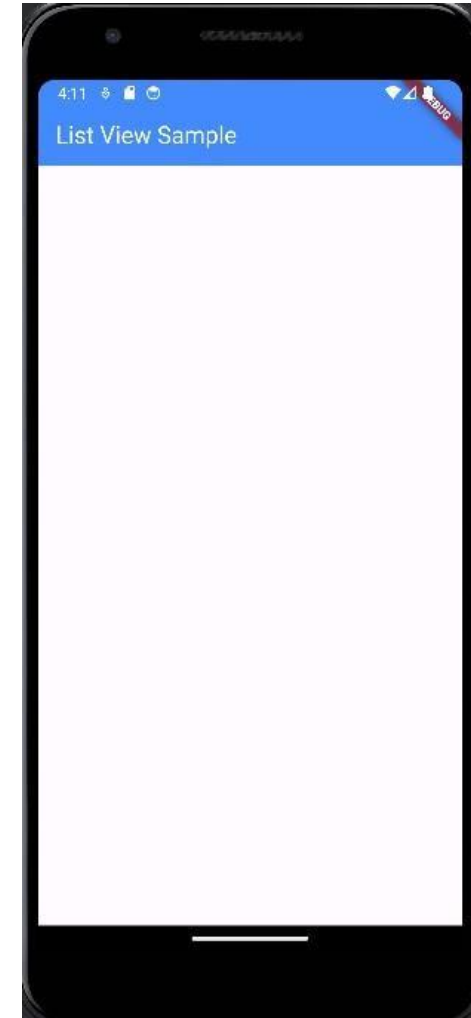
```
@override
Widget build(context){
    contacts = Contact.getAllContacts();

    return Scaffold(
);
}
```

Example App (Contact List)

- Add an AppBar to the Scaffold layout.
- Set the background to blue accent and the text color to white.

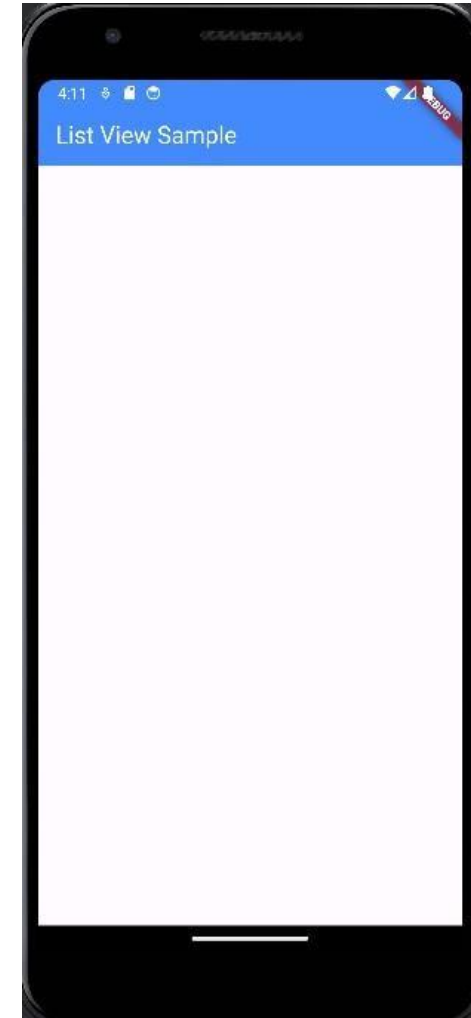
```
return Scaffold(  
  appBar: AppBar(  
    title: const Text(  
      'List View Sample',  
      style: TextStyle(  
        color: Colors.white,  
      ) // TextStyle  
    ), // Text  
    backgroundColor: Colors.blueAccent,  
  ), // AppBar  
); // Scaffold
```



Example App (Contact List)

- Set a ListView builder as the widget of the Scaffold body.
- The itemCount is set to the length of the contact list.

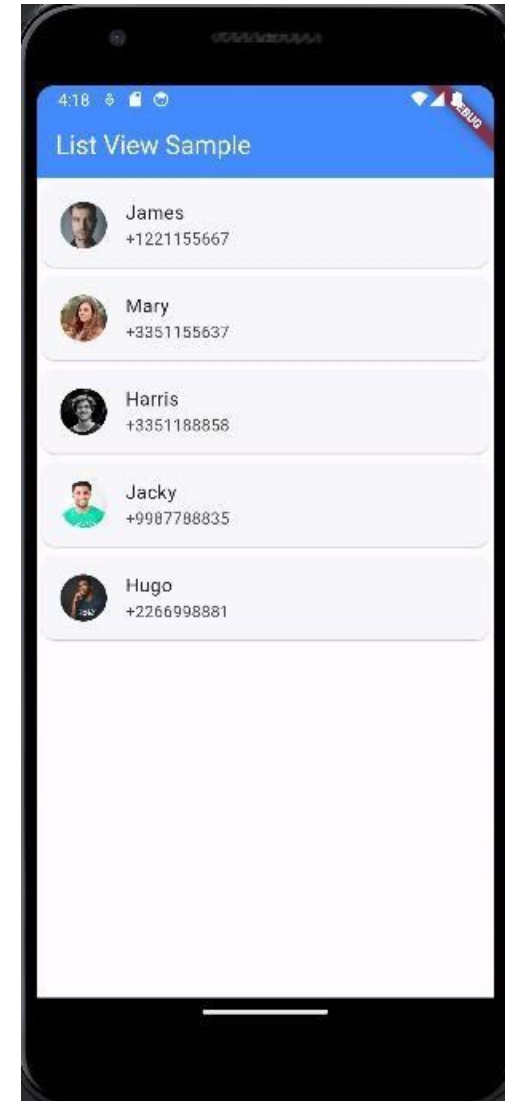
```
return Scaffold(  
  appBar: AppBar(  
    title: const Text(  
      'List View Sample',  
      style: TextStyle(  
        color: Colors.white,  
      ) // TextStyle  
    ), // Text  
    backgroundColor: Colors.blueAccent,  
  ), // AppBar  
  
  body: ListView.builder(  
    itemCount: contacts.length,  
    itemBuilder: (BuildContext context, int index) {  
  
    },  
  ) // ListView.builder  
); // Scaffold
```



Example App (Contact List)

- The itemBuilder should generate a Card Widget which consists of an avatar image, and two text widgets.

```
body: ListView.builder(  
  itemCount: contacts.length,  
  itemBuilder: (BuildContext context, int index) {  
    return Card(  
      color: Colors.white,  
      child: ListTile(  
        leading: CircleAvatar(  
          backgroundImage: AssetImage(  
            contacts[index].image  
          ), // AssetImage  
        ), // CircleAvatar  
        title: Text(contacts[index].name),  
        subtitle: Text(contacts[index].phoneNumber)  
      ) // ListTile  
    ); // Card  
  },  
); // ListView.builder
```

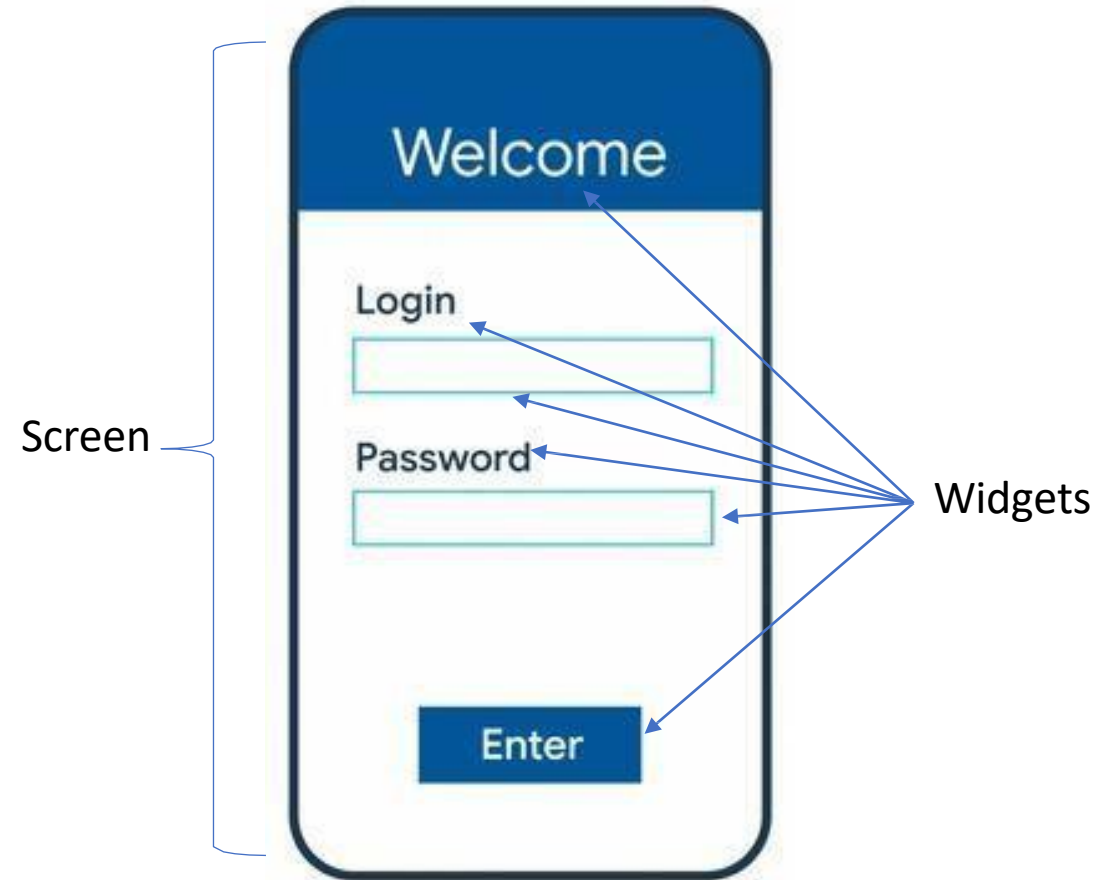


Content

Mobile Application User Interface Operations

- Screens and Widgets
- Working with Multiple Screens
 - Screen navigation with Flutter Navigator

- A widget is a key component of the user interface (UI) in Flutter. Widgets describe the structure and layout of the application's UI elements, such as buttons, text fields, images, containers, and more.
- A screen is a page view (e.g., a login page, a main page, etc.) consisting of multiple widgets.

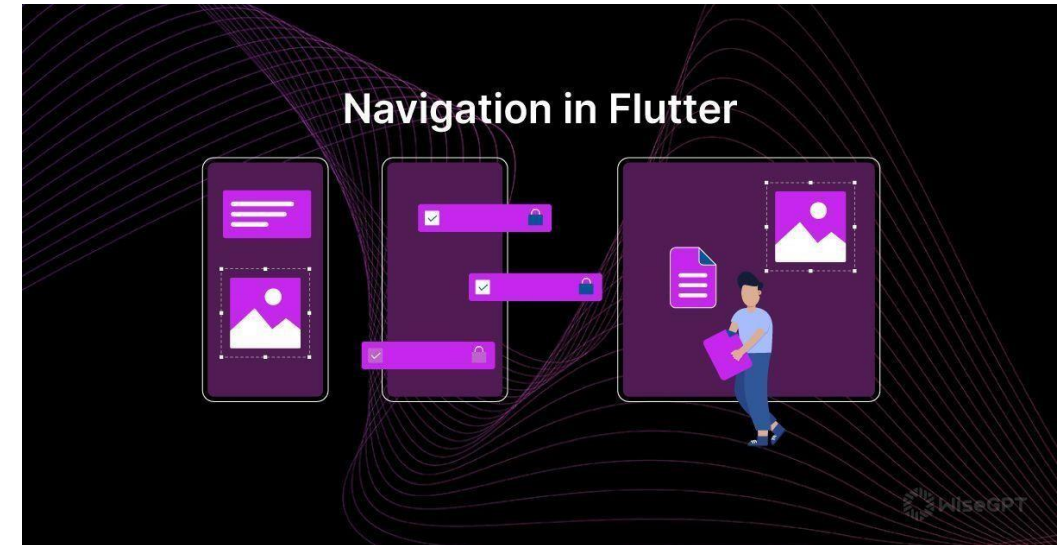


- Navigation in Flutter refers to moving from one screen or page to another within your app.
- It's a fundamental aspect of building mobile applications, enabling users to seamlessly explore different sections or features.



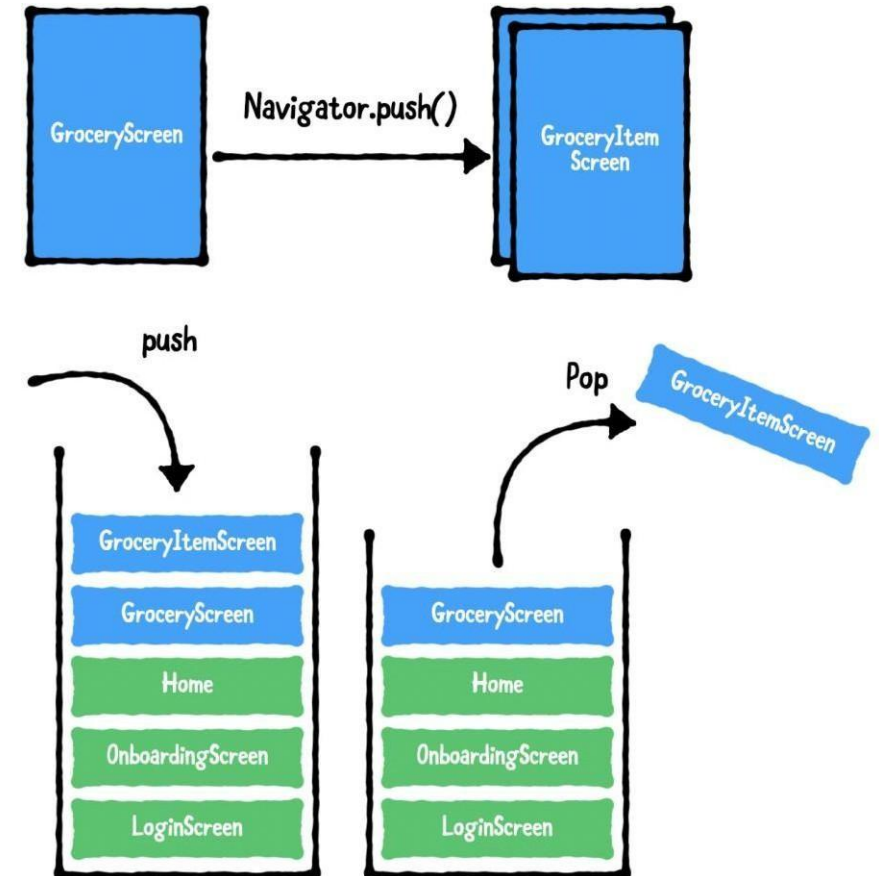
Screen Navigation in Flutter

- Three main types of navigation
 1. Stack Navigation
 2. Tab Navigation
 3. Drawer Navigation



- ## Stack Navigation

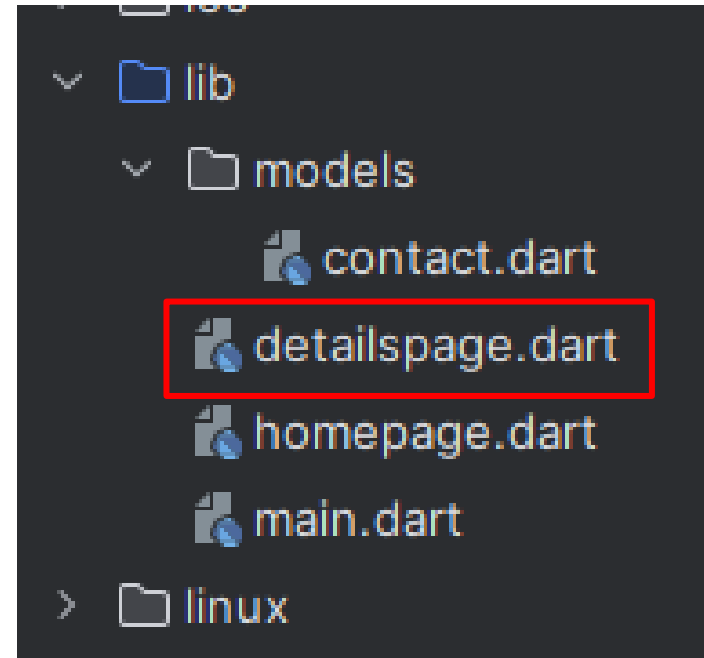
- Imagine a deck of cards, where you can add or remove cards from the top of the stack. Stack Navigation in Flutter works similarly.
- When you move to a new screen, the current screen is pushed onto the navigation stack, and when you return, the top screen is popped off the stack.
- This navigation type is commonly used for hierarchical and linear flows within an app.



CLASS ACTIVITY

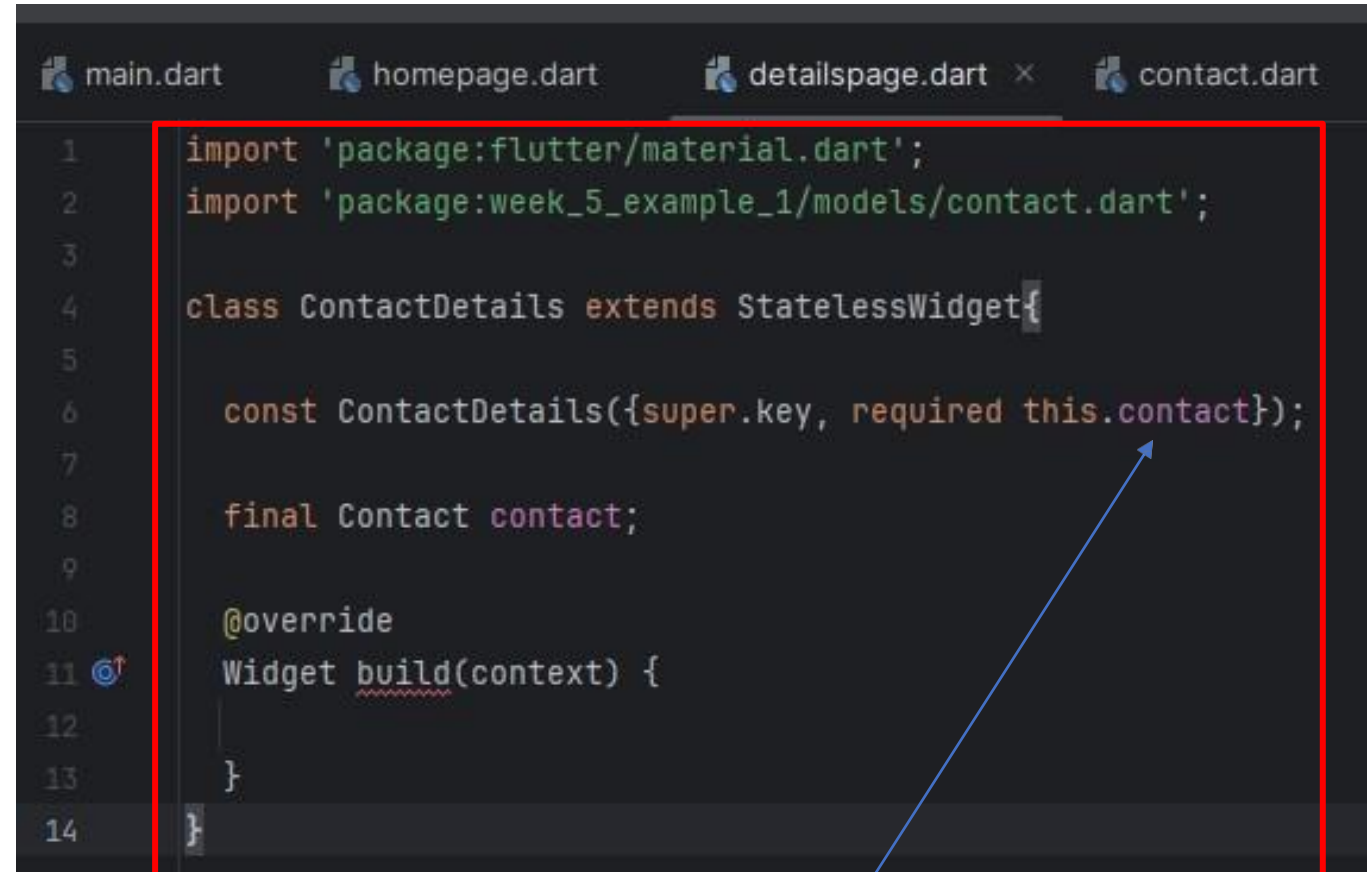
(2nd Hands-On Practice)

- From the existing project from Example 1 app, create a new dart file named “**detailspage.dart**” in the lib folder.



Example 2

- In the detailspage.dart file, create a ContactDetails class that extends a StatelessWidget class.
- The ContactDetails class should accept a parameter named “contact”

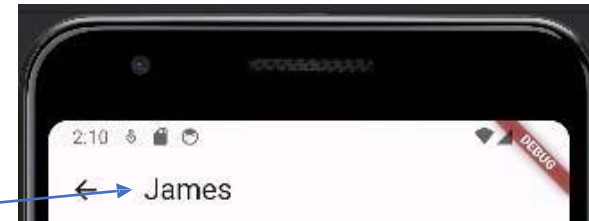


```
1 import 'package:flutter/material.dart';
2 import 'package:week_5_example_1/models/contact.dart';
3
4 class ContactDetails extends StatelessWidget {
5
6     const ContactDetails({super.key, required this.contact});
7
8     final Contact contact;
9
10    @override
11    Widget build(context) {
12
13    }
14 }
```

a parameter named “contact”

- Add a Scaffold layout widget.
- Set the AppBar title as the person's name in the contact.

```
class ContactDetails extends StatelessWidget {  
  
  const ContactDetails({super.key, required this.contact});  
  
  final Contact contact;  
  
  @override  
  Widget build(context) {  
    return Scaffold(  
      appBar: AppBar(title: Text(contact.name)),  
    ); // Scaffold  
  }  
}
```



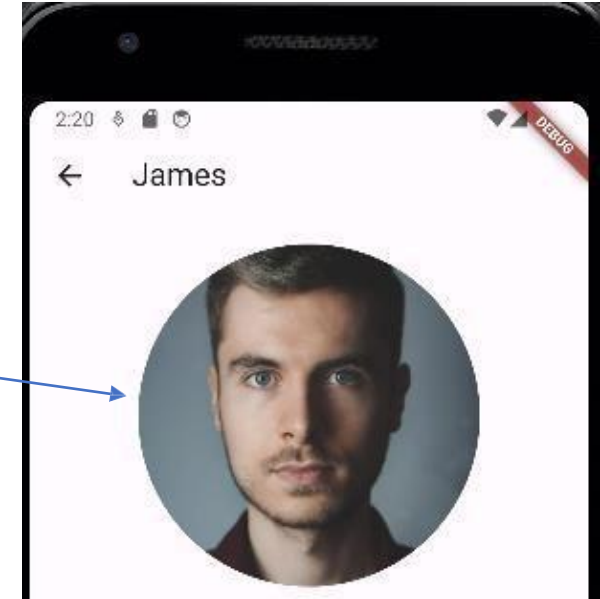
- Add a Padding layout widget to wrap a Container.
- In the Container, set alignment to center so that the widgets in the container will be aligned to the center.
- Set a Column layout widget as the child of the Container.

```
@override
Widget build(context) {
  return Scaffold(
    appBar: AppBar(title: Text(contact.name)),
    body: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Container(
        alignment: Alignment.center,
        child: Column(
          ), // Column
        ), // Container
      ), // Padding
    ); // Scaffold
}
```

Example 2

- In the Column widget, add a Circle Avatar widget as one of the children of the Column.
- Create a SizedBox widget to leave 15 px vertical gap for the next widget.

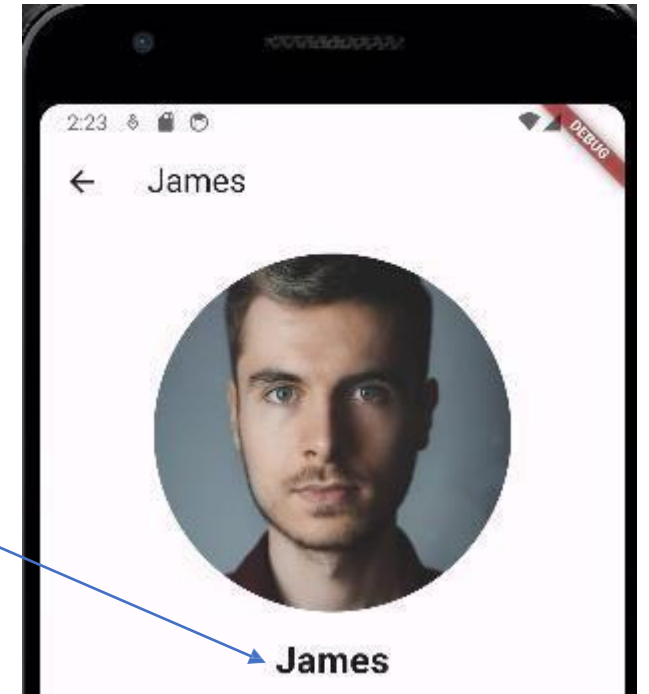
```
body: Padding(  
  padding: const EdgeInsets.all(20.0),  
  child: Container(  
    alignment: Alignment.center,  
    child: Column(  
      children: [  
        CircleAvatar(  
          radius: 120,  
          backgroundImage: AssetImage(contact.image),  
        ), // CircleAvatar  
        const SizedBox(height: 15),  
      ],  
    ), // Column  
  ), // Container  
) // Padding
```



Example 2

- Add a Text widget as part of the Children of the Column.
- This text widget should display the contact name in bold format.
- Add another SizedBox widget to leave 15 px vertical gap.

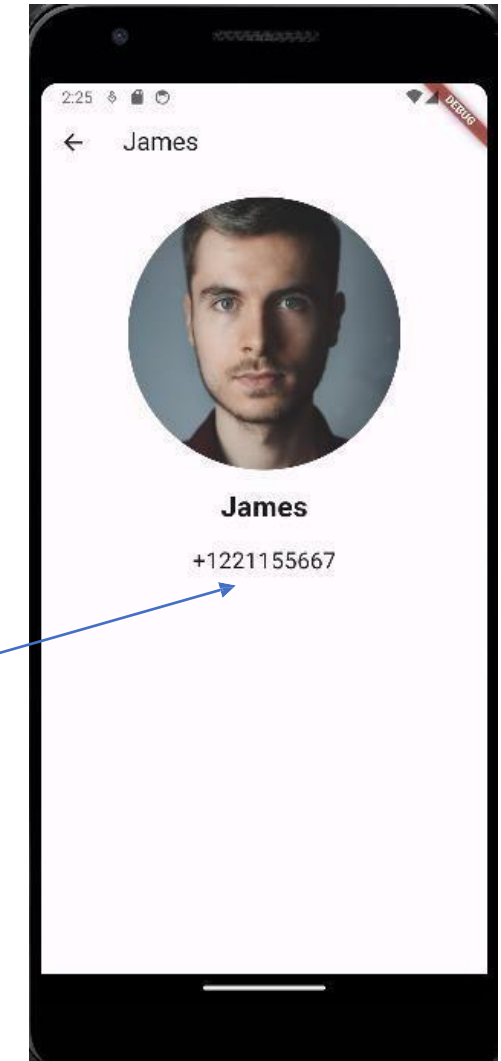
```
child: Column(  
  children: [  
    CircleAvatar(  
      radius: 120,  
      backgroundImage: AssetImage(contact.image),  
    ), // CircleAvatar  
    const SizedBox(height: 15),  
    Text(  
      contact.name,  
      style: const TextStyle(  
        fontSize: 25,  
        fontWeight: FontWeight.bold  
      ), // TextStyle  
    ), // Text  
    const SizedBox(height: 15),  
  ],  
, // Column
```



Example 2

- Add another Text widget as part of the Children of the Column.
- This text widget should display the contact number.

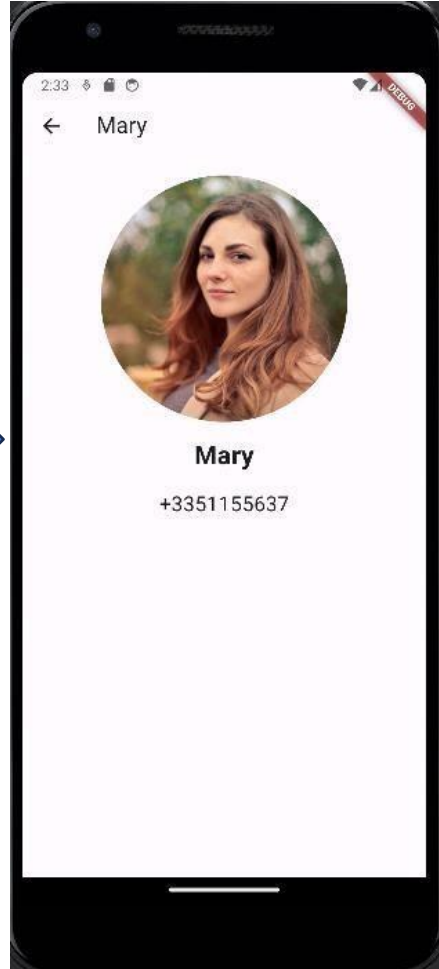
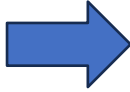
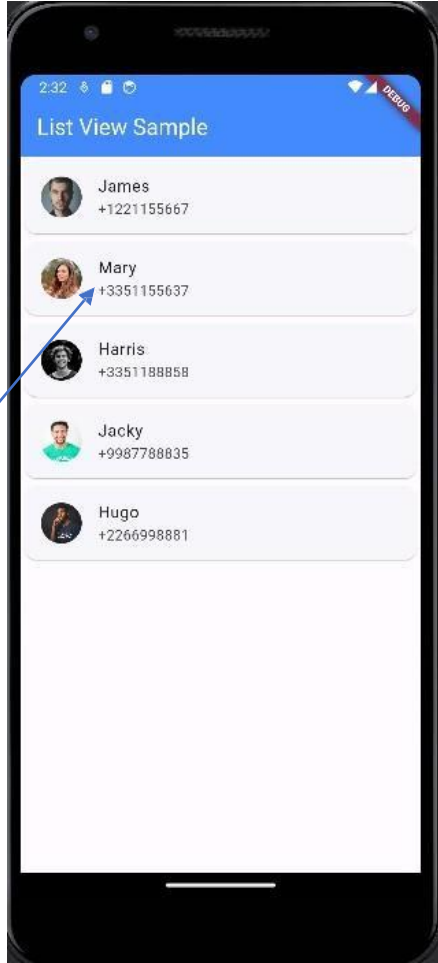
```
const SizedBox(height: 10),  
Text(  
  contact.name,  
  style: const TextStyle(  
    fontSize: 25,  
    fontWeight: FontWeight.bold  
  ), // TextStyle  
) // Text  
const SizedBox(height: 15),  
Text(  
  contact.phoneNumber,  
  style: const TextStyle(  
    fontSize: 20,  
  ) // TextStyle  
) // Text  
],  
) // Column
```



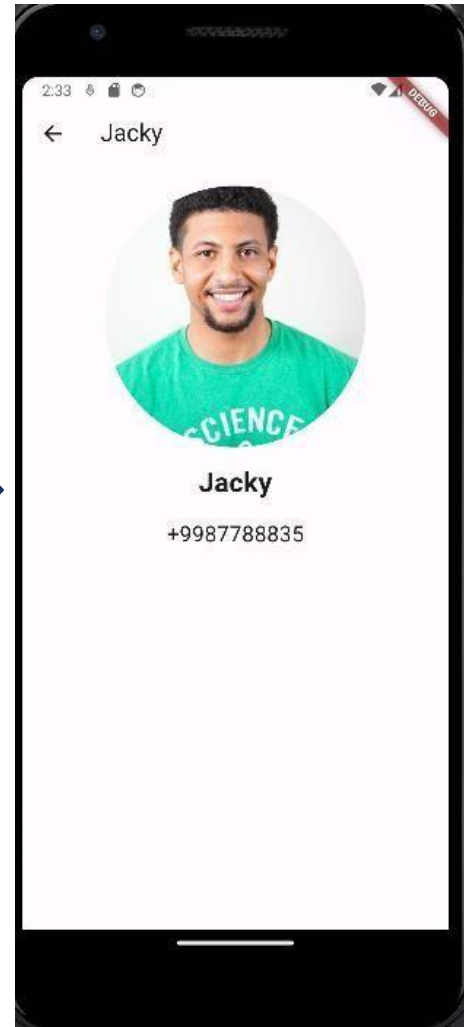
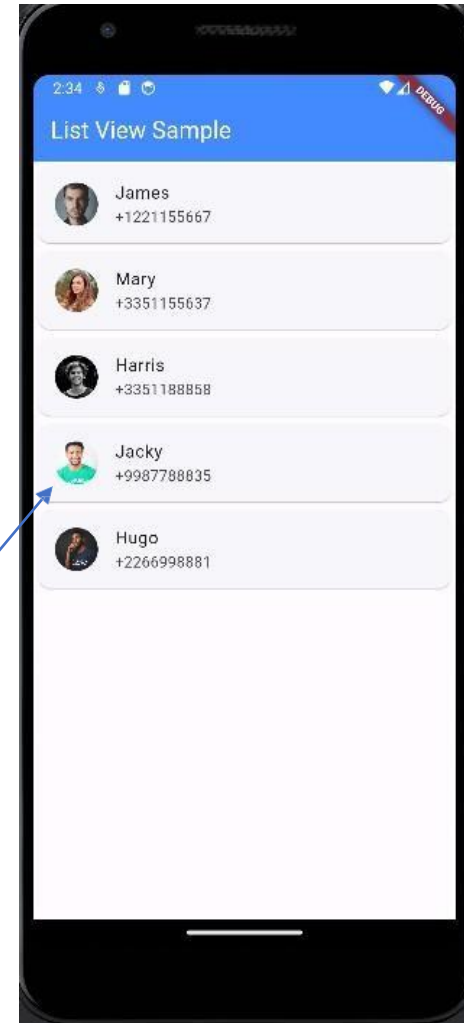
- In the homepage.dart file, add an onTap property to the ListTile.
- Define an anonymous function for onTap.
- This function will use Navigator to route a screen to another screen when user click on one of the item in the list.

```
return Card(  
  color: Colors.white,  
  child: ListTile(  
    leading: CircleAvatar(  
      backgroundImage: AssetImage(  
        contacts[index].image  
      ), // AssetImage  
    ), // CircleAvatar  
    title: Text(contacts[index].name),  
    subtitle: Text(contacts[index].phoneNumber),  
    onTap: () {  
      Navigator.of(context).push(  
        MaterialPageRoute(  
          builder: (context)  
            => ContactDetails(contact: contacts[index])  
        ) // MaterialPageRoute  
      );  
    },  
  ), // ListTile  
); // Card
```

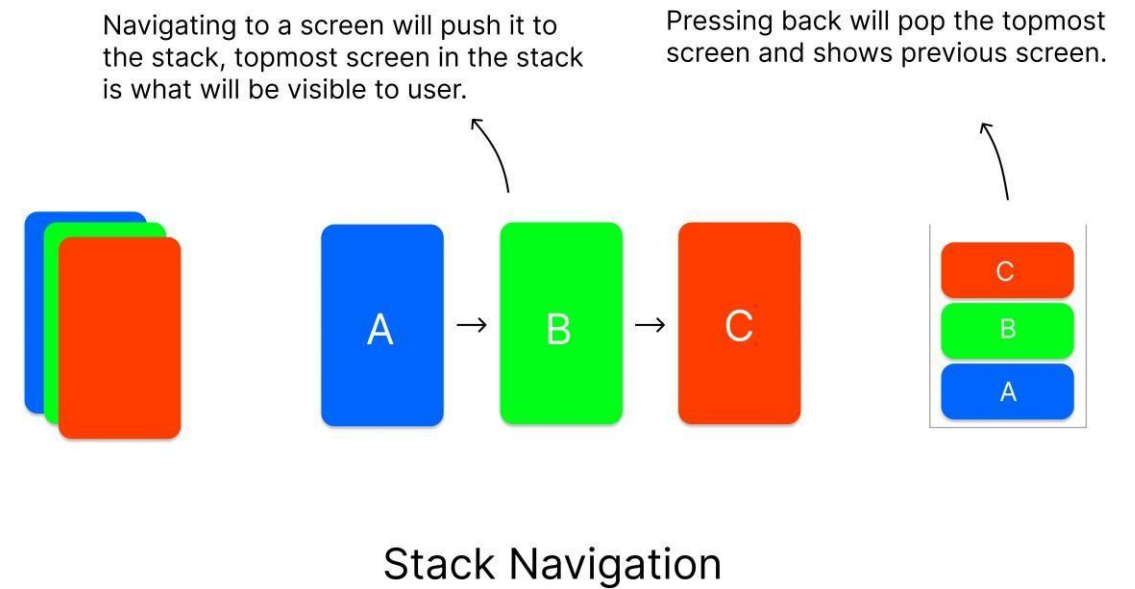
Click



Click



- Navigator is a widget that helps us to navigate between the screens.
- The navigator follows stack method when dealing with the screens.
- Based on the actions made by the user, the screens are stacked one over the other and when pressed back, it goes to the most recently visited screen.



- Flutter provides a basic routing class `MaterialPageRoute` and two methods `Navigator.push()` and `Navigator.pop()` that shows how to navigate between two routes.

```
onTap: (){  
  Navigator.of(context).push(  
    MaterialPageRoute(  
      builder: (context)  
        =>ContactDetails(contact: contacts[index])  
    ) // MaterialPageRoute  
  );  
},
```

A Contact object

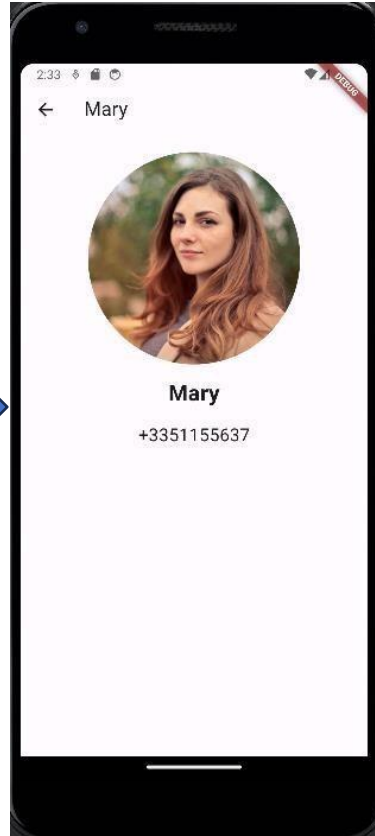
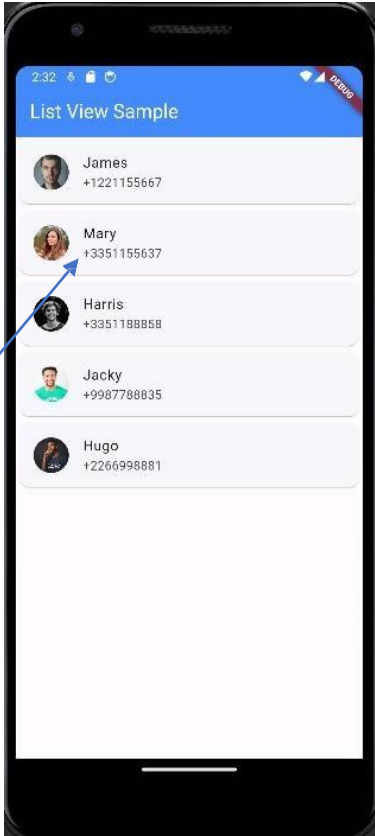


Constructor function that build a Contact Details screen



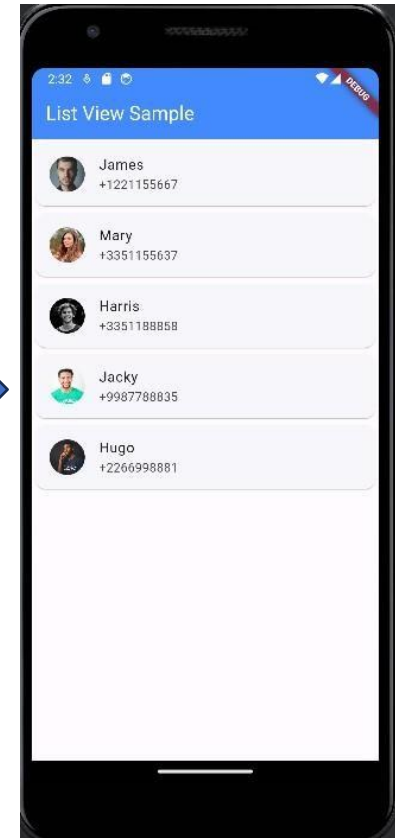
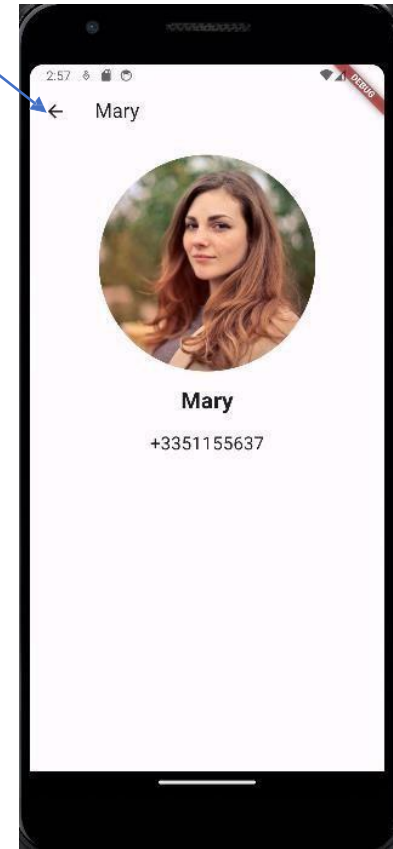
Navigator will
push the
Contact
Details on top
of the stack

Click



Click

Navigator will
automatically
implement
pop method
to remove the
Contact
Details screen
from the stack



See you next class